

Inteligencia Artificial

Resolución del BACP (*Balanced Academic Curriculum Problem*)

mediante Búsqueda Tabú

Oscar Mauricio Ruiz Jimenez

23 de junio de 2026

Evaluación

Mejoras 2da Entrega (10 %):	_____
Código Fuente (10 %):	_____
Representación (5 %):	_____
Descripción del algoritmo (15 %):	_____
Experimentos (10 %):	_____
Resultados (40 %):	_____
Conclusiones (10 %):	_____
Nota Final (100):	_____

Resumen

El *Balanced Academic Curriculum Problem* (BACP) consiste en asignar asignaturas universitarias a semestres equilibrando la carga académica, sujeto a las relaciones de pre-requisito y a las cotas de capacidad semestral. Este informe presenta una solución basada en **Búsqueda Tabú** sobre una representación vectorial, con construcción voraz aleatorizada, movimiento de traslado restringido a la ventana de precedencia, lista tabú dinámica y penalizaciones adaptativas. La propuesta se evalúa sobre ocho instancias del *benchmark* y de mallas reales de la UTFSM, con cinco semillas cada una. Los resultados muestran **soluciones factibles en todas las instancias**, con desbalance nulo o cercano a cero en la mayoría y tiempos inferiores a 1,2 s, en línea con el consenso de la literatura sobre la eficacia de las metaheurísticas de búsqueda local.

1. Introducción

El diseño de mallas curriculares universitarias es una tarea compleja que afecta directamente la experiencia académica de los estudiantes. Una distribución desbalanceada de los cursos puede provocar periodos de saturación extrema alternados con baja actividad, lo que impacta negativamente en el rendimiento y la retención estudiantil. El *Balanced Academic Curriculum Problem* (BACP), introducido por Castro y Manzano [3], modela este desafío como un problema de optimización combinatoria con restricciones.

En el sistema de educación superior chileno la carga académica se mide bajo el estándar SCT-Chile (*Sistema de Créditos Transferibles*), adoptado por las universidades del CRUCH para armonizar el trabajo académico total del estudiante [5]. En particular, la Universidad Técnica Federico Santa María fija desde 2021 un máximo de 35 créditos SCT por semestre por estudiante [11], lo que convierte el diseño de una malla curricular balanceada en una necesidad práctica concreta, y no solo en un problema teórico.

La relevancia de este estudio dentro del área de la Inteligencia Artificial radica en que, debido a su naturaleza NP-Hard y a la alta densidad de sus restricciones de precedencia, los métodos de búsqueda exhaustiva resultan infactibles sobre el BACP. Por ello, la literatura recurre a técnicas de búsqueda local, metaheurísticas y enfoques híbridos que combinan distintos algoritmos. El propósito de este informe es caracterizar formalmente el problema, revisar la literatura especializada e implementar y evaluar una solución basada en Búsqueda Tabú. La contribución principal consiste en sintetizar y contrastar la eficacia de los enfoques de Programación con Restricciones (CP) frente a las metaheurísticas modernas, y en validar empíricamente una propuesta de resolución sobre instancias del *benchmark* y de mallas reales de la UTFSM.

El acercamiento propuesto consiste en una metaheurística de **Búsqueda Tabú** sobre una representación vectorial del problema, que parte de una construcción voraz aleatorizada y explora el espacio mediante un movimiento de traslado con vecindario restringido, apoyándose en una lista tabú dinámica y penalizaciones adaptativas para escapar de óptimos locales. La propuesta se evaluó experimentalmente sobre ocho instancias (del *benchmark* clásico y de mallas reales de la UTFSM) en un entorno GNU/Linux, ejecutando cinco semillas por instancia; los resultados muestran que el algoritmo obtiene soluciones factibles en todas las instancias, con desbalance nulo o cercano a cero en la mayoría de ellas y tiempos inferiores a 1,2 segundos.

El documento se organiza de la siguiente manera: la Sección 2 define el problema, sus dificultades de resolución y principales variantes; la Sección 3 presenta el estado del arte; la Sección 4 detalla la formulación matemática del problema; la Sección 5 describe la representación adoptada; la Sección 6 detalla el algoritmo implementado; las Secciones 7 y 8 presentan los experimentos y sus resultados; finalmente, la Sección 9 recoge las conclusiones y proyecciones del estudio.

2. Definición del Problema

2.1. Descripción general

El BACP consiste en asignar un conjunto de asignaturas $A = \{a_1, a_2, \dots, a_n\}$ a un conjunto de semestres $S = \{1, 2, \dots, N\}$, donde cada asignatura a_i posee una carga académica $w_i \in \mathbb{Z}^+$ y puede tener un conjunto de prerrequisitos $\text{Pre}(a_i) \subseteq A$. La asignación debe respetar las relaciones de prerrequisito (una asignatura sólo puede cursarse en un semestre posterior al de todos sus prerrequisitos) y cumplir con restricciones sobre la carga total y el número de asignaturas por semestre. El objetivo es minimizar el desbalance de carga académica entre los semestres.

2.2. Dificultades y complejidad del problema

La resolución del BACP presenta desafíos computacionales significativos. Dada su estructura matemática, estrechamente relacionada con problemas de empaquetamiento y asignación

generalizada, el BACP está clasificado dentro de la categoría NP-Hard [3]. Esto implica que el tamaño del espacio de soluciones crece de manera exponencial a medida que aumenta el número de asignaturas y semestres. En mallas curriculares extensas, como las de los programas de ingeniería, intentar evaluar todas las combinaciones posibles mediante métodos exactos resulta computacionalmente intratable para instancias de gran escala [4].

Además de la explosión combinatoria, la alta densidad de las restricciones dificulta el proceso de optimización. Las extensas cadenas de prerequisites restringen severamente los vecindarios válidos, generando un espacio de búsqueda altamente fragmentado. Como señalan Chiarandini et al. [4], esta topología provoca que los algoritmos de búsqueda local queden atrapados frecuentemente en óptimos locales, requiriendo mecanismos avanzados de diversificación (como los *kickers* o cadenas de eyección) para lograr transitar hacia regiones factibles y alcanzar un balance de carga equitativo.

2.3. Variantes conocidas

En la literatura se han estudiado diversas variantes del BACP:

- **BACP clásico:** Formulado originalmente por Castro y Manzano [3], donde se minimiza la varianza de la carga académica entre semestres, sujeto a restricciones de prerequisite y cotas de carga y número de asignaturas por semestre.
- **BACP con restricciones adicionales:** Monette et al. [8] extienden el modelo incluyendo restricciones de co-requisito (asignaturas que deben cursarse en el mismo semestre), restricciones de incompatibilidad y restricciones de orden más complejas.
- **BACP revisado:** Chiarandini et al. [4] reformulan el problema identificando inconsistencias en las instancias originales de la literatura, proporcionando un conjunto de instancias revisadas y un *benchmark* actualizado. Esta variante es la más utilizada en trabajos recientes.
- **Multi-objetivo:** Algunos trabajos consideran objetivos adicionales, como la minimización del número de semestres o la maximización del balance entre asignaturas teóricas y prácticas.

2.4. Detalle de la variante a estudiar

En este trabajo se estudia la variante clásica del BACP, siguiendo la formulación de Castro y Manzano, con el conjunto de instancias revisado por Chiarandini et al. El problema se caracteriza por los siguientes parámetros:

- Un conjunto de asignaturas con su respectiva carga académica.
- Relaciones de prerequisite entre asignaturas.
- Número máximo de semestres N .
- Carga académica mínima (C_{min}) y máxima (C_{max}) por semestre.
- Número mínimo (n_{min}) y máximo (n_{max}) de asignaturas por semestre.

La variable de decisión es la asignación de cada asignatura a un semestre específico. El objetivo es minimizar el desbalance de la carga académica entre los semestres, buscando que sea lo más equitativa posible.

Las **variables** del problema son:

- La asignación de cada asignatura a un semestre específico de la malla curricular.

Las **restricciones** que debe cumplir toda solución válida son:

- Cada asignatura debe quedar asignada a exactamente un semestre.
- Si una asignatura es prerrequisito de otra, debe cursarse en un semestre estrictamente anterior.
- La carga académica total de cada semestre debe mantenerse dentro de los límites mínimo y máximo establecidos.
- El número de asignaturas asignadas a cada semestre debe respetar los valores mínimo y máximo definidos.
- La malla curricular debe completarse dentro del número máximo de semestres disponibles.

3. Estado del Arte

3.1. Origen y trabajos fundacionales

El BACP fue introducido formalmente en el año 2001 por Castro y Manzano [3] en el contexto de la planificación académica automatizada. Originalmente, el problema surgió como un caso de estudio para evaluar estrategias de ordenamiento de variables y valores en problemas de satisfacción de restricciones (CSP). En su trabajo fundacional, los autores modelaron el problema como un CSP y estudiaron heurísticas de selección de variables (*fail-first*, *smallest domain*) y de ordenamiento de valores (*min-conflicts*) para mejorar la eficiencia de la búsqueda. Este trabajo proporcionó el conjunto de instancias de referencia original, que se convirtió en el *benchmark* de la comunidad durante la década siguiente.

3.2. Enfoques de Programación con Restricciones (CP)

El enfoque más estudiado para el BACP es la programación con restricciones (CP), que permite modelar el problema de forma natural mediante variables, dominios y restricciones, y resolverlo mediante algoritmos de búsqueda con propagación de restricciones.

Monette y su equipo de investigación [8] presentaron un modelo CP detallado para el BACP. Sus principales contribuciones incluyen: una nueva heurística de ramificación, el uso de relaciones de dominancia, experimentos con varios criterios de balance, y una propuesta híbrida de Programación con Restricciones y Búsqueda Local. Adicionalmente, Schaus y su equipo de investigación, co-autores de esa investigación, desarrollaron en trabajos paralelos la restricción global *spread* para modelar directamente el objetivo de balance en problemas como el BACP.

3.3. Metaheurísticas y búsqueda local

Para abordar la complejidad de las instancias generalizadas del BACP, los enfoques más eficaces de la última década se apoyan en el marco de las Máquinas de Búsqueda Local Generalizada (GLSM), formalizado originalmente por Hoos y Stützle [7] en el contexto de la búsqueda local estocástica y aplicado al BACP primero por Di Gaspero y Schaerf en 2008 [6] y, de forma extensiva, por Chiarandini et al. en 2012 [4]. En este marco, el proceso de optimización se apoya en componentes de búsqueda clasificados en dos categorías principales: exploradores continuos o *runners* y saltos complejos o *kickers*.

A continuación, se detallan las principales estrategias empleadas en este enfoque:

- **Recorrido Simulado (SA):** Actúa como un *runner* probabilístico. En cada paso de la búsqueda, se selecciona un vecino aleatorio. Si el movimiento mejora la solución actual, se acepta de inmediato. Si resulta en una solución peor, con una degradación de calidad igual a Δf , el movimiento aún puede ser aceptado con una probabilidad de $e^{-\Delta f/T}$, donde T representa el parámetro de decrecimiento temporal, llamado *temperatura*. Se emplea un esquema de enfriamiento geométrico donde, tras evaluar una cantidad definida de movimientos, la temperatura decrece mediante un factor $\gamma < 1$ (es decir, $T \leftarrow \gamma T$) [4].
- **Búsqueda Tabú Dinámica (DTS):** Implementada extensamente por Chiarandini et al. [4], opera como un *runner* agresivo que explora un subconjunto del vecindario y adopta la mejor solución posible, independientemente de si mejora o empeora el costo actual. Para evitar ciclos, los movimientos recientes ingresan a una lista tabú a corto plazo durante un número aleatorio de iteraciones en el rango $[k_{min}, k_{max}]$. Este estatus puede ser anulado por un criterio de aspiración si el movimiento genera una nueva mejor solución global. Además, las penalizaciones de las restricciones se adaptan dinámicamente mediante un factor ξ para guiar la búsqueda hacia regiones factibles o permitir mayor exploración [4].
- **Saltos Complejos (K):** Los *kickers* realizan un único salto de gran envergadura secuenciando múltiples movimientos elementales, emulando cadenas de eyección. Para mantener la eficiencia, solo se evalúan secuencias lógicamente vinculadas, como alteraciones en un mismo período o en cursos con currículos compartidos. Estos saltos pueden ser aleatorios (K_r) para diversificación, o enfocarse en la intensificación adoptando la primera secuencia de mejora (K_f) o la mejor secuencia absoluta tras una exploración exhaustiva (K_b) [4]. Cabe destacar que esta estructura de kickers fue introducida originalmente por Di Gaspero y Schaerf [6] y retomada por Chiarandini para su aplicación al BACP [4].
- **Optimización por Colonia de Hormigas (ACO):** Rubio et al. [10] presentaron la primera aplicación de ACO al BACP mediante el modelo *Best-Worst Ant System* (BWAS). Este enfoque refuerza feromonas en aristas pertenecientes a buenas soluciones, penaliza las de la peor solución actual, e incorpora mutación de rastros de feromonas para introducir diversidad y evitar el estancamiento de la búsqueda.
- **Algoritmo de Luciérnagas Discreto (DFA):** Al-Obeidat et al. [1] propusieron una versión discreta del algoritmo de luciérnagas combinada con un mecanismo de búsqueda local (LSM). Las soluciones iniciales se construyen mediante programación lineal y se representan en una matriz binaria, obteniendo resultados competitivos en instancias del CSPLib e instancias reales.

3.4. Representaciones del problema y su efectividad

En la literatura se han utilizado principalmente dos formas de representar computacionalmente el BACP, dependiendo de la técnica de resolución a emplear:

- **Representación Matricial (Binaria):** Utilizada comúnmente en métodos exactos como la Programación Lineal Entera (ILP). Consiste en una matriz de variables binarias $x_{ij} \in \{0, 1\}$ que indica si la asignatura i se imparte en el semestre j .
- **Representación Vectorial (Entera):** Utilizada mayoritariamente en metaheurísticas. Consiste en un vector de asignación $x = (x_1, x_2, \dots, x_n)$, donde cada componente $x_i \in \{1, \dots, N\}$ representa el semestre asignado a la asignatura a_i [4].

En los enfoques de búsqueda local aplicados al BACP, la representación vectorial es la más utilizada. Chiarandini et al. [4] definen una solución candidata bajo esta representación al diseñar sus heurísticas, mientras que los métodos exactos como ILP emplean naturalmente la representación matricial. Su principal ventaja radica en que garantiza estructuralmente la restricción

dura de “asignación única” (una asignatura no puede estar en dos semestres al mismo tiempo), lo que reduce drásticamente el tamaño del espacio de búsqueda. De este modo, los algoritmos pueden concentrarse exclusivamente en optimizar el balance de carga, empleando los siguientes movimientos de vecindad fundamentales:

- **Intercambio (*Swap*):** Intercambio de semestres entre dos asignaturas a_i y a_j , validando la factibilidad de los prerrequisitos.
- **Traslado (*Move*):** Reasignación de una asignatura a_i a un semestre diferente.
- **Movimientos Compuestos (*Kickers*):** Consisten en una secuencia de movimientos elementales del vecindario relacionados entre sí, operando bajo una lógica similar a las cadenas de eyección (*ejection chains*) [4]. Al evaluar la secuencia completa como una sola acción, permiten realizar cambios estructurales profundos en la malla curricular.

3.5. Manejo de restricciones

En la resolución del BACP mediante búsqueda local, la literatura especializada aborda el manejo de restricciones separándolas en dos categorías: restricciones duras (*hard constraints*) y restricciones blandas (*soft constraints*).

Por un lado, la factibilidad de los prerrequisitos suele tratarse como una restricción dura. Para mantener la coherencia cronológica de la malla curricular, los algoritmos limitan su exploración a vecindarios que no violen las precedencias, apoyándose teóricamente en los algoritmos híbridos para CSP descritos por Prosser [9], los cuales podan movimientos infactibles antes de ejecutarlos.

Por otro lado, las restricciones de capacidad semestral (carga académica y número de cursos) se manejan mediante un enfoque de reparación con restricciones blandas. Según describen Di Gaspero y Schaerf [6] y aplican Chiarandini y su equipo [4], la Búsqueda Tabú Dinámica (DTS) está equipada con un mecanismo que adapta dinámicamente la forma de la función de costo. Si una restricción de capacidad se satisface durante un número determinado de iteraciones, su peso de penalización (P_i) se relaja reduciéndolo por un factor ξ ($P_i \leftarrow P_i/\xi$), lo que permite al algoritmo explorar temporalmente regiones infactibles del espacio de búsqueda. Por el contrario, si la restricción es violada, la penalización se endurece ($P_i \leftarrow P_i \cdot \xi$) con el objetivo de forzar a la búsqueda a retornar hacia la factibilidad.

3.6. Síntesis de resultados y eficacia algorítmica

En cuanto a la eficacia de los algoritmos desarrollados hasta la fecha, la literatura establece un claro consenso. Los enfoques iniciales basados puramente en Programación con Restricciones (CP) propuestos por Castro y Manzano, si bien fueron pioneros, resultan insuficientes para encontrar el óptimo global en instancias reales de gran tamaño. De manera similar, los métodos exactos de Programación Lineal Entera (ILP) logran resolver instancias pequeñas, pero fracasan en tiempos de ejecución aceptables al escalar el problema.

En contraste, las metaheurísticas basadas en Búsqueda Local se posicionan actualmente como los algoritmos más eficaces. Específicamente, el enfoque de Máquinas de Búsqueda Local Generalizada (GLSM) implementado por Chiarandini et al. [4] logró encontrar soluciones factibles de alta calidad para todas las instancias del *benchmark* revisado, acercándose significativamente a los límites teóricos (*lower bounds*) en tiempos computacionales razonables, superando así a los enfoques tradicionales.

A modo de síntesis, el Cuadro 1 consolida y contrasta los principales enfoques revisados, indicando sus autores y año, la representación empleada, el manejo de restricciones y su eficacia relativa. De esta comparación se desprende una tendencia clara: el campo evolucionó desde métodos exactos (CP e ILP), precisos pero limitados a instancias pequeñas, hacia metaheurísticas de búsqueda local —en particular la Búsqueda Tabú Dinámica— que constituyen el estado del

arte por resolver instancias reales de gran tamaño con alta calidad y en tiempos razonables, en tanto que las propuestas bioinspiradas más recientes (ACO y *Firefly*) se validan principalmente sobre las instancias clásicas del CSPLib.

Enfoque	Autores (año)	Repres.	Manejo de restricciones	Eficacia / escalabilidad
CP	Castro y Manzano (2001)	Binaria	Duras con propagación	Pionero; óptimo solo en instancias pequeñas
CP híbrido + LS	Monette et al. (2007)	Binaria	Restricción global <i>spread</i>	Mejora la búsqueda; instancias clásicas
DTS (GLSM)	Di Gaspero y Schaefer (2008); Chiarandini et al. (2012)	Vectorial	Duras (vecindario) + blandas (penalización dinámica ξ)	Estado del arte: factible y de alta calidad en todas las instancias revisadas
ACO (BWAS)	Rubio et al. (2013)	Vectorial	Penalización	Competitivo; evita el estancamiento
DFA discreto	Al-Obeidat et al. (2023)	Matriz binaria	LP inicial + búsqueda local (LSM)	Competitivo en CSPLib e instancias reales

Cuadro 1: Síntesis comparativa de los principales enfoques de resolución del BACP.

3.7. Tendencias actuales y enfoques recientes

Las investigaciones más recientes sobre el BACP se han enfocado en superar las limitaciones de las instancias clásicas, explorando nuevas metaheurísticas inspiradas en la naturaleza e integrando el problema a escenarios académicos más grandes.

Por un lado, se ha documentado el éxito de algoritmos basados en inteligencia de enjambres para evadir óptimos locales en el BACP. Ejemplos destacados de esto son la aplicación de la Optimización por Colonia de Hormigas (ACO) implementada por Rubio et al. [10], así como enfoques discretos más recientes basados en el algoritmo de las luciérnagas (*Firefly Algorithm*) [1], los cuales han demostrado una convergencia rápida en mallas curriculares complejas.

Por otro lado, la tendencia macro en la investigación, como detallan Bettinelli et al. [2], apunta hacia la resolución de problemas integrados de planificación de horarios académicos (*curriculum-based course timetabling*). En estos enfoques modernos, el balance de la malla curricular deja de ser un problema aislado y se convierte en la etapa precursora indispensable antes de realizar la asignación física de profesores, bloques horarios y salas de clases.

4. Modelo Matemático

El modelo matemático presentado a continuación se basa en la formulación original de Castro y Manzano [3], incorporando las correcciones y el formalismo propuesto por Chiarandini et al. [4] en su versión revisada del problema.

4.1. Parámetros

De acuerdo a la estructura del problema, se definen los siguientes parámetros:

- A : Conjunto de todas las asignaturas que deben ser asignadas.
- w_i : Carga académica asociada a cada asignatura $i \in A$.

- P : Conjunto de pares de prerrequisitos, donde $(a, b) \in P$ implica que la asignatura a debe cursarse antes que b .
- N : Número máximo de semestres disponibles en la malla curricular.
- C_{min}, C_{max} : Carga académica mínima y máxima permitida por semestre.
- n_{min}, n_{max} : Número mínimo y máximo de asignaturas permitidas por semestre.
- $\bar{C}$: Carga académica semestral promedio.

4.2. Variable de Decisión

Se define una variable binaria para representar la asignación de las asignaturas a los periodos académicos:

$$x_{ij} = \begin{cases} 1 & \text{si la asignatura } i \in A \text{ se asigna al semestre } j \in \{1, \dots, N\} \\ 0 & \text{en caso contrario} \end{cases}$$

donde $x_{ij} \in \{0, 1\}$, $\forall i \in A, \forall j \in \{1, \dots, N\}$.

4.3. Función Objetivo

El objetivo es minimizar el desbalance de la carga académica entre los semestres, buscando que sea lo más equilibrada posible respecto al promedio:

$$\text{mín } f(x) = \sum_{j=1}^N (C_j - \bar{C})^2$$

Donde C_j es la carga total del semestre j , calculada como $C_j = \sum_{i \in A} w_i \cdot x_{ij}$.

4.4. Restricciones

El modelo está sujeto a las siguientes condiciones de factibilidad:

1. **Asignación única:** Cada asignatura debe ser asignada a exactamente un semestre.

$$\sum_{j=1}^N x_{ij} = 1 \quad \forall i \in A$$

2. **Respeto de prerrequisitos:** Si una asignatura a es prerrequisito de b , el semestre de a debe ser estrictamente menor al de b .

$$\sum_{j=1}^N j \cdot x_{aj} < \sum_{j=1}^N j \cdot x_{bj} \quad \forall (a, b) \in P$$

3. **Límites de carga semestral:** La carga total de cada periodo debe mantenerse dentro de los rangos definidos.

$$C_{min} \leq \sum_{i \in A} w_i \cdot x_{ij} \leq C_{max} \quad \forall j \in \{1, \dots, N\}$$

4. **Límites de cantidad de asignaturas:** El número de cursos por semestre debe cumplir con los mínimos y máximos académicos.

$$n_{min} \leq \sum_{i \in A} x_{ij} \leq n_{max} \quad \forall j \in \{1, \dots, N\}$$

5. Representación

Para la resolución del BACP mediante Búsqueda Tabú se adopta la **representación vectorial entera**, que es la más utilizada en los enfoques de búsqueda local de la literatura [4]. Una solución se describe como un vector

$$x = (x_1, x_2, \dots, x_n), \quad x_i \in \{1, \dots, N\},$$

donde la componente x_i indica el semestre asignado a la asignatura a_i . De este modo, un único vector describe la malla curricular completa.

5.1. Interpretación y ejemplo

Cada posición del vector corresponde a una asignatura y su valor al semestre en que se imparte. La carga de un semestre j se obtiene sumando los créditos de las asignaturas cuyo valor es j . El Cuadro 2 ilustra una instancia reducida de seis asignaturas y la solución $x = (1, 1, 2, 2, 1, 2)$: las asignaturas a_1, a_2, a_5 se cursan en el semestre 1 y a_3, a_4, a_6 en el semestre 2, produciendo cargas de 12 y 13 créditos respectivamente.

i	Asignatura	Créditos	Prerreq.	x_i
1	Cálculo I	5	–	1
2	Álgebra	4	–	1
3	Cálculo II	5	a_1	2
4	Física	4	a_1	2
5	Programación	3	–	1
6	Estructuras	4	a_5	2

Cuadro 2: Ejemplo de representación vectorial. La solución $x = (1, 1, 2, 2, 1, 2)$ respeta los prerrequisitos ($x_1 = 1 < x_3 = 2$, etc.) y produce cargas semestrales de 12 y 13.

5.2. Justificación

La representación vectorial es adecuada para el problema por varias razones. En primer lugar, **garantiza estructuralmente la restricción de asignación única**: como cada asignatura posee exactamente una componente, es imposible representar una asignatura en dos semestres, lo que reduce drásticamente el espacio de búsqueda respecto de la representación matricial binaria x_{ij} . En segundo lugar, existe una relación *uno a uno* entre cada vector y una asignación completa de la malla, y la solución óptima siempre es representable. En tercer lugar, es **informativa y se complementa con el operador de movimiento**: modificar una sola componente reasigna una asignatura y altera la carga de a lo sumo dos semestres, lo que permite evaluar el efecto de un movimiento de forma eficiente. Finalmente, permite transitar tanto por el espacio factible como por el infactible: las violaciones de capacidad se penalizan en la función de evaluación en lugar de prohibirse, lo que habilita a la búsqueda a atravesar regiones infactibles para escapar de óptimos locales.

6. Descripción del Algoritmo

La técnica implementada es una **Búsqueda Tabú** inspirada en la *Dynamic Tabu Search* (DTS) de Chiarandini et al. [4]. La búsqueda parte de una solución inicial construida mediante una heurística voraz aleatorizada y la mejora iterativamente explorando un vecindario de movimientos elementales, manteniendo una lista tabú de movimientos recientes para escapar

de óptimos locales. El proceso se repite desde varias soluciones iniciales (multi-arranque) y se reporta la mejor solución encontrada.

6.1. Estructura general

El ciclo principal de la Búsqueda Tabú se resume en el siguiente pseudocódigo:

```

ENTRADA: instancia, parametros (max_iter, max_sin_mejora,
    kmin, kmax, alpha)
SALIDA : mejor solucion encontrada

x      <- solucion_inicial_greedy_aleatorizada(alpha)
mejor  <- x
w[s]   <- P_base   para todo semestre s   # pesos de penalizacion
sin_mejora <- 0
para iter = 1 .. max_iter mientras sin_mejora < max_sin_mejora:
    mejor_mov <- ninguno
    para cada asignatura i:
        [lo, hi] <- ventana de precedencia de i
        para cada semestre s en [lo, hi], con s != x_i:
            d <- variacion de costo al mover i hacia s
            si (mov no es tabu) o (cumple aspiracion):
                conservar el movimiento de menor d
    aplicar mejor_mov a x
    marcar tabu el movimiento inverso por un numero
        aleatorio de iteraciones en [kmin, kmax]
    actualizar pesos w[s] (endurecer si s viola una cota,
        relajar si la cumple)
    si costo_estable(x) < costo_estable(mejor):
        mejor <- x ; sin_mejora <- 0
    sino:
        sin_mejora <- sin_mejora + 1
retornar mejor

```

El procedimiento anterior corresponde a un *arranque*. El algoritmo completo ejecuta este procedimiento R veces (parámetro **restarts**), cada una desde una solución inicial distinta, y devuelve la mejor solución global. Este esquema de multi-arranque favorece la **diversificación**, mientras que la selección del mejor vecino en cada iteración aporta **intensificación**.

6.2. Función de evaluación

La calidad de una solución se mide combinando el objetivo de balance con penalizaciones por violar las restricciones blandas de capacidad:

$$f(x) = \sum_{j=1}^N (C_j - \bar{C})^2 + \sum_{j=1}^N P_j \cdot v_j(x),$$

donde el primer término es el desbalance (suma de desviaciones cuadradas de la carga C_j respecto del promedio $\bar{C}$), $v_j(x)$ es la magnitud de violación de las cotas de carga y de cantidad de asignaturas en el semestre j , y P_j es su peso de penalización. Las violaciones de prerrequisito no aparecen en f porque se tratan como **restricción dura** mediante el vecindario (Sección 6.4).

El peso de penalización es **dinámico** (mecanismo ξ de la DTS): cada semestre tiene su propio peso P_j que se *endurece* ($P_j \leftarrow P_j \cdot \xi$) mientras viola una cota y se *relaja* ($P_j \leftarrow P_j / \xi$) cuando

la cumple, acotado inferiormente por un valor base P_{base} . Así, un semestre persistentemente sobrecargado incrementa su penalización hasta que la búsqueda se ve forzada a repararlo, lo que mejora la convergencia hacia la factibilidad. Para que las comparaciones entre iteraciones sean consistentes pese a los pesos variables, la mejor solución y el criterio de aspiración se evalúan con una métrica *estable* de peso fijo P_{base} . Los parámetros son $P_{base} = 1000$ (suficientemente mayor que el desbalance máximo factible, de modo que toda solución factible domina a cualquier infactible) y $\xi = 1,5$.

6.3. Inicialización: greedy topológico aleatorizado

La solución inicial se construye con una heurística voraz que respeta las precedencias y que incorpora **aleatoriedad** (atendiendo la retroalimentación de Estado de Avance, ver Sección 6.6):

```
calcular nivel[i] = camino mas largo desde una fuente (prereqs)
calcular cola[i]  = camino mas largo hacia un sumidero (sucesores)
ordenar asignaturas por nivel ascendente,
    barajando el orden dentro de cada nivel
para cada asignatura i en ese orden:
    lo <- max(nivel[i], 1 + max semestre de los prereqs ya ubicados)
    hi <- N - cola[i] # deja sitio a los sucesores
    candidatos <- semestres en [lo, hi] que respetan las cotas maximas
    RCL <- la fraccion 'alpha' de candidatos con menor carga actual
    x_i <- un semestre elegido al azar dentro de la RCL
```

El nivel ordena las asignaturas según sus dependencias; la *cola* acota por arriba la ubicación de cada asignatura para que toda su cadena de sucesores quepa dentro de los N semestres, lo que garantiza que la solución inicial **nunca viola prerequisites**. La aleatoriedad proviene de dos fuentes: el barajado dentro de cada nivel y la selección dentro de una *Lista Restringida de Candidatos* (RCL) de estilo GRASP. El parámetro $\alpha \in [0, 1]$ controla el tamaño de la RCL: con α pequeño la construcción es casi voraz (poca diversidad), y con α grande es más aleatoria (mayor diversidad entre arranques).

6.4. Movimiento: traslado (*Move*) con vecindario restringido

El único operador de variación es el **traslado** (*Move*): reasignar una asignatura a_i desde su semestre actual a otro semestre,

$$x_i : s_{\text{actual}} \rightarrow s_{\text{nuevo}}.$$

Un cambio en x_i altera la carga de a lo sumo dos semestres, por lo que la variación de la función de evaluación se calcula de forma incremental en $O(1)$ amortizado.

El vecindario de *Move* se **restringe a la ventana de precedencia** de cada asignatura (atendiendo la retroalimentación de Estado de Avance, Sección 6.6). Para la asignatura i los destinos válidos son los semestres $s \in [lo_i, hi_i]$ con

$$lo_i = \max\left(1, \max_{p \in \text{Pre}(a_i)} (x_p + 1)\right), \quad hi_i = \min\left(N, \min_{q: a_i \in \text{Pre}(a_q)} (x_q - 1)\right),$$

es decir, después del último de sus prerequisites y antes del primero de sus sucesores. De este modo **ningún movimiento generado puede violar una precedencia**, en lugar de generar candidatos infactibles y descartarlos a posteriori; esto reduce el tamaño del vecindario y concentra la búsqueda en optimizar el balance y la capacidad. A modo de ejemplo, dado $x = (1, 1, 2, 2, 1, 2)$, mover a_4 (Física) de $s = 2$ a $s = 3$ produce $x = (1, 1, 2, 3, 1, 2)$, movimiento válido si 3 pertenece a su ventana.

6.5. Lista tabú, aspiración e intensificación/diversificación

Tras aplicar un movimiento $x_i : a \rightarrow b$, se prohíbe el movimiento *inverso* (devolver a_i al semestre a) durante un número aleatorio de iteraciones en el rango $[k_{min}, k_{max}]$; esta tenencia aleatoria evita ciclos y favorece la diversificación. El **criterio de aspiración** anula el estatus tabú de un movimiento si éste genera una solución mejor que la mejor global conocida. En cada iteración se adopta el mejor vecino admisible aunque empeore el costo actual (estrategia de *mejor mejora*), lo que constituye el mecanismo de intensificación. La diversificación se logra de forma combinada mediante la lista tabú, el multi-arranque desde soluciones iniciales aleatorizadas y las penalizaciones dinámicas, que reconfiguran el paisaje de búsqueda para escapar de óptimos locales. Los parámetros del operador son $k_{min} = 5$ y $k_{max} = 10$.

6.6. Retroalimentación – Estado de Avance

En la presentación de Estado de Avance se solicitaron dos aspectos a abordar, ambos incorporados en esta entrega:

1. **Agregar aleatoriedad al greedy.** La construcción inicial dejó de ser determinista: se baraja el orden de las asignaturas dentro de cada nivel y se selecciona el semestre mediante una RCL de estilo GRASP (parámetro α). Esto genera soluciones iniciales distintas en cada arranque y habilita el multi-arranque.
2. **Restringir el vecindario del movimiento.** El operador *Move* dejó de explorar todos los semestres; ahora se limita a la ventana de precedencia $[lo_i, hi_i]$, lo que evita generar candidatos que violan prerequisites y reduce considerablemente la proporción de soluciones infactibles evaluadas.

7. Experimentos

7.1. Objetivos y entorno

La experimentación persigue tres objetivos: (i) verificar que el algoritmo alcanza soluciones **factibles** y de bajo desbalance en todas las instancias; (ii) estudiar el efecto del parámetro α (aleatoriedad del greedy) sobre la calidad; y (iii) analizar la **convergencia** de la búsqueda (calidad en función del presupuesto de iteraciones y del tiempo). Los experimentos se ejecutaron en un entorno **GNU/Linux (Ubuntu sobre WSL)** con el compilador **g++ 11.4** y optimización **-O2**, sobre un equipo con procesador **Intel Core i5-11400H** y **16 GB** de memoria RAM. El indicador de calidad es el desbalance $f(x) = \sum_j (C_j - \bar{C})^2$ de la mejor solución factible; se reporta además la factibilidad y el tiempo de ejecución.

7.2. Instancias

Se utilizan las **ocho instancias** provistas: tres del *benchmark* clásico revisado (**bacp8**, **bacp10**, **bacp12**) y cinco correspondientes a mallas reales de la UTFSM. El Cuadro 3 resume sus características. Cubren un rango amplio de tamaño (23 a 67 asignaturas) y de densidad de precedencias (16 a 93 prerequisites), siendo **utfsm2311** la más restringida.

7.3. Diseño experimental

Para dotar de soporte estadístico a los resultados, cada configuración se ejecutó con **5 semillas aleatorias** distintas (semillas 1 a 5). Los parámetros por defecto son: **restarts** = 30, **max_iter** = 50000 por arranque, **max_sin_mejora** = 10000, $k_{min} = 5$, $k_{max} = 10$, $\alpha = 0,3$, $P_{base} = 1000$ y $\xi = 1,5$. El **criterio de término** de cada arranque es alcanzar **max_iter**

Instancia	$ A $	N	#Prereq.	$[C_{min}, C_{max}]$	$[n_{min}, n_{max}]$
bacp8	46	8	38	$[10, 24]$	$[2, 10]$
bacp10	42	10	34	$[10, 24]$	$[2, 10]$
bacp12	66	12	65	$[10, 24]$	$[2, 10]$
utfsm3	23	4	16	$[20, 35]$	$[1, 8]$
utfsm2311	67	12	93	$[20, 35]$	$[1, 8]$
utfsm2920	46	8	29	$[20, 35]$	$[1, 8]$
utfsm7310	57	10	43	$[20, 35]$	$[1, 8]$
utfsm7313	59	11	51	$[20, 35]$	$[1, 8]$

Cuadro 3: Características de las instancias: número de asignaturas $|A|$, semestres N , prerrequisitos, y cotas de carga y de cantidad por semestre.

iteraciones o bien `max_sin_mejora` iteraciones consecutivas sin mejorar la mejor solución. Se realizaron tres experimentos: (E1) ejecución de las ocho instancias con parámetros por defecto; (E2) comparación de $\alpha \in \{0,1,0,3,0,6,0,9\}$ sobre tres instancias representativas; y (E3) análisis de convergencia variando `max_iter` $\in \{1000, 5000, 20000, 50000, 200000\}$ sobre la instancia más difícil.

8. Resultados

8.1. Resultados generales (E1)

El Cuadro 4 presenta, para cada instancia, el mejor y el promedio del desbalance sobre las 5 semillas, la cantidad de ejecuciones factibles y el tiempo promedio. El algoritmo alcanzó **soluciones factibles en las 40 ejecuciones** (8 instancias $\times$ 5 semillas), incluyendo la instancia más restringida (`utfsm2311`). El desbalance es muy bajo en la mayoría de los casos: `utfsm7310` alcanza el balance perfecto ($f = 0$) y `utfsm3` casi perfecto ($f = 0,75$). Los tiempos se mantienen por debajo de 1,2 s incluso en las instancias mayores, lo que evidencia la eficiencia del cálculo incremental del costo.

Instancia	$ A $	N	Mejor f	f promedio	Factibles	t prom. (s)
bacp8	46	8	1.875	1.875	5/5	0.57
bacp10	42	10	2.40	2.40	5/5	0.69
bacp12	66	12	2.00	3.20	5/5	1.01
utfsm3	23	4	0.75	0.75	5/5	0.16
utfsm2311	67	12	17.0	46.6	5/5	1.07
utfsm2920	46	8	2.00	2.00	5/5	0.72
utfsm7310	57	10	0.00	1.20	5/5	0.96
utfsm7313	59	11	5.64	8.84	5/5	0.94

Cuadro 4: Resultados generales (E1): 5 semillas por instancia, parámetros por defecto.

La dificultad relativa de las instancias se correlaciona con la densidad de precedencias y la holgura de capacidad. Las instancias de cargas pequeñas ($[10, 24]$) y muchas opciones de ubicación (`bacp8`, `utfsm3`) son resueltas casi sin variabilidad entre semillas, mientras que `utfsm2311` —la de mayor número de prerrequisitos y cota inferior de carga más exigente— presenta el mayor desbalance y la mayor dispersión entre semillas, lo que indica un paisaje de búsqueda más fragmentado (*Los tiempos corresponden al promedio de las 5 semillas; en utfsm7310 se descartó una medición atípica de 3.8 s atribuible a carga del sistema operativo*).

8.2. Efecto del parámetro α (E2)

El Cuadro 5 muestra el desbalance promedio (sobre 5 semillas, siempre factible) para distintos valores de α . El efecto es **dependiente de la instancia**: en las instancias menos restringidas (**bacp12**, **utfsm7313**) un valor moderado ($\alpha = 0,3$) ofrece el mejor compromiso, mientras que en la instancia más difícil (**utfsm2311**) un valor alto ($\alpha = 0,9$) reduce notablemente el desbalance promedio (de 46,6 a 20,6). Esto confirma que una mayor aleatoriedad en la construcción diversifica los puntos de partida y resulta beneficiosa cuando el espacio de búsqueda es más accidentado, validando la utilidad de la aleatoriedad incorporada al greedy.

Instancia	$\alpha = 0,1$	$\alpha = 0,3$	$\alpha = 0,6$	$\alpha = 0,9$
bacp12	5.20	3.20	5.60	3.20
utfsm2311	57.0	46.6	47.4	20.6
utfsm7313	9.64	8.84	11.24	9.24

Cuadro 5: Desbalance promedio (5 semillas) según el parámetro α de la RCL del greedy (E2).

8.3. Análisis de convergencia (E3)

El Cuadro 6 reporta el desbalance promedio y el tiempo al variar el presupuesto de iteraciones por arranque sobre **utfsm2311**. La calidad mejora de forma marcada al pasar de 1000 a 20000 iteraciones (de 90,2 a 46,6), tras lo cual se **estabiliza**: con 50000 y 200000 iteraciones el resultado es idéntico al de 20000, ya que el criterio de **max.sin.mejora** detiene los arranques al dejar de haber mejoras. El tiempo crece con el presupuesto hasta saturarse en torno a 1,1 s. Se observa, además, que la **factibilidad se alcanza incluso con presupuestos pequeños** (1000 iteraciones), y que el esfuerzo adicional se invierte en refinar el balance. Esto evidencia un buen comportamiento de convergencia: rendimientos decrecientes y un punto de saturación claro.

max_iter	1000	5000	20000	50000	200000
f promedio	90.2	53.4	46.6	46.6	46.6
t prom. (s)	0.10	0.49	1.07	1.09	1.08

Cuadro 6: Convergencia en **utfsm2311**: desbalance y tiempo promedio (5 semillas) según el presupuesto de iteraciones por arranque (E3).

8.4. Comparación con el estado del arte

La literatura establece que las metaheurísticas de búsqueda local, y en particular la DTS de Chiarandini et al. [4], logran **soluciones factibles de alta calidad para todas las instancias** del *benchmark* revisado, acercándose a las cotas inferiores teóricas en tiempos razonables, superando a los enfoques exactos (CP e ILP) que no escalan. Los resultados obtenidos son consistentes con ese consenso: la implementación propuesta, basada en los mismos principios (representación vectorial, movimiento *Move*, lista tabú dinámica y penalizaciones adaptativas), alcanza factibilidad en la totalidad de las instancias y semillas, con desbalance nulo o cercano a cero en varias de ellas, y en tiempos inferiores a 1,2 s. Cabe notar que la comparación numérica directa con algunos trabajos no es inmediata, pues parte de la literatura optimiza la carga máxima por semestre en lugar de la suma de desviaciones cuadradas utilizada en esta formulación.

9. Conclusiones

Del estudio del estado del arte del BACP se extraen las siguientes conclusiones relevantes.

Sobre las técnicas y el problema que resuelven. No todas las técnicas revisadas abordan exactamente la misma variante del problema. Los enfoques de Programación con Restricciones (CP) propuestos por Castro y Manzano [3] y Monette et al. [8] se centran en la variante clásica del BACP con instancias del CSPLib. En contraste, Chiarandini et al. [4] abordan una variante generalizada con instancias reales más complejas, mientras que trabajos recientes como los de Rubio et al. [10] y Al-Obeidat et al. [1] retoman las instancias clásicas del CSPLib para validar nuevas metaheurísticas bioinspiradas.

Similitudes y diferencias entre las técnicas. Todas las técnicas comparten el objetivo de minimizar el desbalance de carga académica respetando las restricciones de prerequisito. Sin embargo, difieren significativamente en su enfoque: los métodos exactos (CP e ILP) garantizan optimalidad pero escalan mal ante instancias grandes, mientras que las metaheurísticas sacrifican la garantía de óptimo global a cambio de soluciones de alta calidad en tiempos razonables. En cuanto a la representación, los métodos exactos emplean variables binarias x_{ij} , mientras que las metaheurísticas de búsqueda local prefieren la representación vectorial por su eficiencia computacional. Respecto al manejo de restricciones, todas las técnicas tratan los prerequisitos como restricciones duras, pero difieren en las restricciones de capacidad: los métodos exactos las imponen estrictamente, mientras que las metaheurísticas las manejan mediante penalizaciones dinámicas.

Sobre la propuesta implementada y sus resultados. La Búsqueda Tabú desarrollada confirma, a escala de este trabajo, el consenso de la literatura: una representación vectorial combinada con un movimiento elemental y una lista tabú dinámica basta para alcanzar la factibilidad en la totalidad de las instancias y semillas evaluadas, con desbalance nulo o cercano a cero en la mayoría de ellas y tiempos inferiores a 1,2 s. Los experimentos arrojan tres conclusiones relevantes. Primero, la calidad de la solución inicial es determinante: acotar la construcción voraz para respetar la cadena de sucesores (que la solución inicial nunca viole prerequisitos) resultó decisivo para que el vecindario restringido pudiera reparar la capacidad y alcanzar factibilidad en las instancias más rígidas. Segundo, el efecto de la aleatoriedad del greedy (α) es dependiente de la instancia: en la más restringida (*utfsm2311*) una mayor diversificación mejora claramente el resultado, lo que concuerda con el supuesto de la formulación de que un espacio de búsqueda más fragmentado exige mayor exploración. Tercero, el análisis de convergencia muestra rendimientos decrecientes y un punto de saturación claro, evidenciando que el algoritmo converge rápidamente y que la factibilidad se logra incluso con presupuestos pequeños. Una observación contraria a la intuición inicial fue que el movimiento *Move*, por sí solo, basta para resolver todas las instancias sin recurrir a operadores compuestos (*kickers*), siempre que la construcción inicial sea de buena calidad.

Limitaciones, trabajo futuro e ideas propias. Una limitación transversal a todas las técnicas revisadas es la dependencia de la calidad de los parámetros de entrada: si la carga académica asignada a cada asignatura no refleja con precisión el trabajo real del estudiante, el algoritmo puede considerar óptima una malla curricular que en la práctica resulta desbalanceada. Esta brecha entre la carga teórica y la carga real percibida por los estudiantes constituye una limitación fundamental del modelo que ningún algoritmo de optimización puede corregir por sí solo.

Como trabajo futuro, resultaría prometedor incorporar mecanismos de retroalimentación basados en datos históricos de rendimiento estudiantil para ajustar dinámicamente las cargas académicas, acercando el modelo matemático a la realidad. Adicionalmente, la extensión del BACP hacia variantes multi-objetivo que consideren simultáneamente el balance de carga y la distribución de tipos de asignaturas representa una dirección de investigación con alto impacto práctico, especialmente en el contexto de mallas curriculares basadas en competencias como las del sistema SCT-Chile.

10. Uso de LLMs

En el desarrollo de este proyecto se utilizaron herramientas de Inteligencia Artificial como apoyo para la redacción y estructuración del documento, bajo el siguiente detalle:

1. **Herramientas utilizadas:** Gemini (versión Advanced/Pro) y Claude (Anthropic, versión gratuita).
2. **Propósito de uso:** Estructurar el formato del documento en LaTeX, mejorar el estilo de redacción académica, corregir gramática y redactar borradores para secciones descriptivas (Estado del Arte y Modelo Matemático) en base a apuntes propios.
3. **Prompts relevantes:** Se utilizaron instrucciones de iteración y rigor académico, como “*Basándote estrictamente en los artículos originales, explica el manejo de restricciones con terminología formal de optimización*” o “*Genera el código LaTeX del modelo matemático a partir de las restricciones y la función objetivo indicadas*”.
4. **Nivel de edición:** Se conservó gran parte de la base sintáctica y gramatical generada por los LLMs, especialmente la estructura del código LaTeX. Sin embargo, no hubo un “copiar y pegar” a ciegas: el contenido generado fue iterado varias veces para alinear la redacción algorítmica estricta con las metodologías reales de los autores.
5. **Validación:** Toda afirmación técnica, modelo matemático y cita bibliográfica generada por la IA fue verificada rigurosamente acudiendo a las fuentes primarias. Se contrastó la información directamente con las diapositivas de la asignatura y los artículos originales de la literatura (*Chiarandini et al., 2012; Monette et al., 2007; Castro y Manzano, 2001*).

Referencias

- [1] Feras Al-Obeidat et al. Balancing academic curriculum problem solution: A discrete firefly-based approach. *Education Research International*, 2023.
- [2] Andrea Bettinelli, Valentina Cacchiani, Roberto Roberti, and Paolo Toth. An overview of curriculum-based course timetabling. *TOP: An Official Journal of the Spanish Society of Statistics and Operations Research*, 23(2):313–349, 2015.
- [3] C. Castro and S. Manzano. Variable and value ordering when solving balanced academic curriculum problems. In *Proceedings of the 7th International Conference on Principles and Practice of Constraint Programming (CP 2001)*. Springer, 2001.
- [4] M. Chiarandini, L. Di Gaspero, S. Gualandi, and A. Schaerf. The balanced academic curriculum problem revisited. *Journal of Heuristics*, 18(1):119–148, 2012.
- [5] Consejo de Rectores de Universidades Chilenas. Manual para la implementación del sistema de créditos académicos transferibles SCT-Chile. http://sct-chile.consejoderectores.cl/documentos_WEB/Sistema_de_creditos_transferibles/manual_sct/Manual_para_la_Implementacion_del_SCT-Chile_2_edicion.pdf, 2013. Último acceso: Mayo 2026.
- [6] L. Di Gaspero and A. Schaerf. A multi-neighbourhood local search algorithm for the balanced academic curriculum problem. In *Proceedings of the 7th International Workshop on the Practice and Theory of Automated Timetabling (PATAT 2008)*, 2008.
- [7] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations and Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Morgan Kaufmann, San Francisco, CA, USA, 2004.

- [8] J.-N. Monette, P. Schaus, S. Zampelli, Y. Deville, and P. Dupont. A CP approach to the balanced academic curriculum problem. In *Proceedings of the 4th International Workshop on Local Search Techniques in Constraint Satisfaction (LSCS 2007)*, 2007.
- [9] Patrick Prosser. Hybrid algorithms for the constraint satisfaction problem. *Computational Intelligence*, 9(3):268–299, 1993.
- [10] José Rubio, Ricardo Soto, Broderick Crawford, et al. Solving the balanced academic curriculum problem using the ACO algorithm. *Mathematical Problems in Engineering*, 2013.
- [11] Universidad Técnica Federico Santa María. Preguntas frecuentes — dirección de estudios. <https://dee.usm.cl/wp-content/uploads/2023/01/Preguntas-Frecuentes-2023.pdf>. Último acceso: Mayo 2026.